SUBNEXA

Intelligent Subscription Management
& Recurring Billing System

HACKATHON PROJECT SPECIFICATION

AI-ready product requirements, technical architecture, role-based UI/UX, data model, workflows and implementation plan.

Project type	Full-stack SaaS / business automation
Recommended build	React + FastAPI + PostgreSQL
Architecture	Modular monolith with background billing worker
Target users	Admin, Staff, Customer
Document purpose	Hand-off specification for an AI coding agent and hackathon team

Hackathon strategy: Build a polished, working end-to-end MVP first. Prioritize subscription creation, reliable recurring invoice generation, payment tracking, customer isolation and role-specific dashboards. Add advanced integrations only after the core workflow works.

Prepared as a development blueprint. Product and technical choices are recommendations for a hackathon implementation; validate the final stack against event rules and available infrastructure.

1. Executive Summary

SubNexa is a centralized platform for businesses that sell recurring software subscriptions, add-ons and products with variants. It replaces spreadsheet-based subscription tracking, manually remembered renewals, manually prepared invoices and payment matching with a consistent digital workflow.

The platform manages products and plans, quotations, subscriptions, billing periods, invoices, discounts, taxes, payments and operational reporting. It provides separate workspaces for Admin, Staff and Customers.

1.1 Problem being solved

- Subscription and renewal details are scattered across spreadsheets.
- Renewals can be missed because dates are remembered manually.
- Invoices are created manually and may contain pricing or tax errors.
- Payments are manually matched against bank statements.
- Management lacks a reliable view of active subscriptions, received revenue and overdue balances.

1.2 Project objectives

- Centralize subscription-business operations in one application.
- Automate invoice generation for subscriptions due for billing.
- Prevent duplicate or skipped billing periods.
- Give each user role only the pages and records it is allowed to access.
- Provide clear dashboards and reports for operational and financial decisions.
- Keep the system practical to build, demo and extend after the hackathon.

1.3 Hackathon MVP scope

Must build for demo	Can be an extension / future scope
Login and role-based access	Online payment gateway
Products, variants and plans	Multi-currency and multi-company
Create and activate subscriptions	Proration and usage-based billing
Generate invoices from due subscriptions	Advanced revenue recognition
Record manual and partial payments	Automated card collection
Admin, Staff and Customer dashboards	Complex dunning campaigns
Invoice PDF and basic notifications	External accounting integrations

2. Hackathon-Optimized Technology Stack

For a time-boxed hackathon, use a conventional, fast-to-develop full-stack architecture rather than making Odoo the core framework. Odoo is powerful, but a custom Odoo addon introduces edition/version dependencies and a steeper setup path. The recommended stack gives the team direct control over the UI, APIs and demo flow.

2.1 Recommended stack

Layer	Technology	Reason
Frontend	React 18 + Vite	Fast setup, component ecosystem, easy demo iteration
UI styling	Tailwind CSS + shadcn/ui	Consistent professional UI with reusable components
Icons	Lucide React	Consistent lightweight icon set
Charts	Recharts	Dashboard charts and visual summaries
Backend	Python + FastAPI	Typed APIs, validation and automatic API documentation
ORM	SQLAlchemy	Structured database models and transactions
Migrations	Alembic	Version-controlled database schema
Database	PostgreSQL	Relational integrity and transactional billing
Authentication	JWT access/refresh + Argon2id	Role-aware authentication and password security
Background jobs	APScheduler for MVP; Celery + Redis if needed	Scheduled billing and renewal checks
PDF	ReportLab	Generate invoice PDFs
Email	SMTP provider	Password reset and invoice notifications
Deployment	Docker Compose	Repeatable local demo and deployment
Testing	Pytest + HTTPX + Playwright	API, business logic and UI testing

2.2 Why not use Odoo as the main hackathon stack?

- A custom Odoo solution is viable when the event expects ERP customization or the team already knows Odoo.
- Odoo edition, version and module availability can affect subscription and accounting features.
- For a short hackathon, React + FastAPI makes the role-specific SaaS interface and demo flow easier to tailor.
- The project can still adopt Odoo-inspired concepts: ORM-backed business models, role groups, record-level access, scheduled actions, invoice workflows and audit trails.

Odoo alternative (only if required by the event): Odoo + PostgreSQL + Python custom addons + OWL + XML/QWeb + Odoo ORM + security groups + scheduled actions. Do not combine this with a separate FastAPI backend for the same business logic.

2.3 Infrastructure rule

Use PostgreSQL as the source of truth. Redis is optional and only needed if Celery or caching is introduced. Do not use Firebase, Supabase or another backend-as-a-service. Do not store financial state only in browser storage.

3. System Architecture

Start with a modular monolith: one backend application divided into clear business modules. This is faster to build and easier to debug than microservices during a hackathon.

```
Architecture flow
Admin / Staff / Customer
    ↓
React web application (Vite, Tailwind, shadcn/ui)
    ↓ HTTPS / REST API
FastAPI application (authentication, permissions, business services)
    ↓ SQLAlchemy + transactions
PostgreSQL (users, products, subscriptions, invoices, payments)
    ■ Scheduled billing worker / email service / PDF generator
```

3.1 Backend modules

- auth: registration, login, refresh, logout and password reset.
- users: roles, staff management and account status.
- customers: customer profiles and ownership relationships.
- catalog: products, variants, plans, taxes and discounts.
- quotations: quotation templates, quotation lines and validity.
- subscriptions: subscription lifecycle and subscription lines.
- billing: due-period detection, billing records and invoice creation.
- invoices: invoice lines, totals, status and PDF generation.
- payments: payment records, allocations and balances.
- analytics: dashboard aggregates and filtered reports.
- notifications: email events and delivery status.
- audit: trace important administrative and financial actions.

3.2 Suggested repository structure

```
subnexa/
■■■ frontend/
■ ■■■ src/{components,layouts,pages,services,hooks,context}
■■■ backend/
■ ■■■ app/{api,core,models,schemas,services,repositories}
■ ■■■ workers/{billing,renewals,notifications}
■ ■■■ migrations/
■ ■■■ tests/
■■■ docker-compose.yml
■■■ .env.example
■■■ README.md
```

3.3 Backend request flow

API router → authentication → role/ownership authorization → Pydantic validation → service layer → SQLAlchemy transaction → PostgreSQL. Keep business rules in services, not React components.

4. User Roles and Permissions

The application has three primary roles. Enforce authorization on every protected backend endpoint; hiding a frontend menu item is not a security control.

Capability	Admin	Staff	Customer
View Admin dashboard	Yes	No	No
Manage staff accounts	Yes	No	No
Manage products and plans	Full	View / limited	No
Create quotations	Yes	Yes	No
Manage subscriptions	All	Permitted records	Own records / allowed requests
Manage invoices	All	Permitted records	Own invoices: view/download
Record payments	Yes	Yes	No
Configure taxes/discounts	Yes	No by default	No
View financial analytics	Full	Limited	Own account only
System settings / audit	Yes	No	No

4.1 Admin

Owns configuration and business oversight: staff accounts, catalogue, plans, tax/discount rules, subscriptions, billing, invoices, payments, analytics and system settings.

4.2 Staff

Handles daily operations: customers, quotations, subscriptions, invoices and manual payment recording. Staff cannot create staff accounts or change protected global settings unless explicitly granted permission.

4.3 Customer

Uses a separate portal to view their own subscriptions, renewal dates, invoices, payment history and profile. Customer data access must be constrained by authenticated customer ownership on the server.

4.4 Core security tests

- A customer cannot retrieve another customer's invoice by changing an ID in the URL.
- A Staff user cannot create another Staff or Admin account.
- A Customer cannot call Admin APIs directly.
- Internal cost prices and audit data are not exposed to customers.
- Every sensitive action is checked on the backend and recorded where appropriate.

5. UI/UX Design System

Create a polished SaaS interface with consistent navigation and component behavior. Admin and Staff use an operational sidebar layout; Customers use a simplified self-service portal.

5.1 Brand and colors

Token	Color	Hex / usage
Primary	Indigo	#5B5FEF – primary actions, active links
Primary hover	Dark indigo	#484BC7 – hover state
Sidebar	Deep navy	#111827 – internal navigation
Page background	Light gray	#F7F8FC – app canvas
Card	White	#FFFFFF – cards and forms
Main text	Dark navy	#172033 – headings/body
Secondary text	Slate	#64748B – hints and metadata
Border	Light gray	#E5E7EB – dividers and outlines
Success	Green	#16A34A – paid/active/success
Warning	Amber	#D97706 – due/attention
Danger	Red	#DC2626 – overdue/failed/cancelled
Info	Blue	#2563EB – informational status

5.2 Typography and dimensions

Element	Specification
Font family	Inter; fallback system-ui, Arial, sans-serif
Page title	28 px / 700
Section title	20 px / 600
Card title	16 px / 600
Body	14 px / 400
Table header	13 px / 600
Helper text	12 px / 400
KPI value	26–30 px / 700
Page padding	24 px desktop; 16 px mobile
Card padding / radius	20–24 px / 12 px
Input / button height	42 px / 40 px
Sidebar width	About 250 px desktop; collapsible on smaller screens

5.3 Shared layout

- Internal app: left sidebar, 64 px top header, breadcrumb, search, notification icon and profile menu.
- Main content: page title and subtitle, primary action, filters, KPI cards, tables/charts.
- Tables: sticky or clear headers, status badges, search, filters, sorting and pagination.

- Forms: grouped sections, clear labels, inline validation, save/cancel actions.
- Customer portal: simplified top navigation or compact sidebar, account-focused cards and invoice downloads.
- Responsive: tables become horizontally scrollable or card-based on mobile; forms become one column.

6. Page-by-Page Application Specification

The page list below defines the minimum navigation and content required for the hackathon MVP. Pages marked 'MVP' should be implemented and demonstrated; later enhancements can be simplified or deferred.

ID / Page	Role / route	Required content and behavior
01 Login (MVP)	All /login	Email, password, show/hide, forgot password; redirect by role.
02 Register (MVP)	Customer /register	Name, email, password, confirm; unique email; customer-only account.
03 Forgot password	All /forgot-password	Email input; generic success message; secure reset email flow.
04 Reset password	All /reset-password	New password, confirmation, expiry and single-use token.
05 Admin dashboard (MVP)	Admin /admin/dashboard	Active subscriptions, revenue received, outstanding, overdue, customers, renewals; revenue chart, status breakdown, upcoming renewals and activity.
06 Customers (MVP)	Admin /admin/customers	Searchable paginated table; customer detail with subscriptions, invoices and payments.
07 Products (MVP)	Admin /admin/products	Name, SKU, type, sales price, cost price, notes, active state; CRUD/archive.
08 Product variants	Admin /admin/products/:id/variants	Variant attribute, name, additional price and active state; calculate variant price.
09 Plans (MVP)	Admin /admin/plans	Name, price, daily/weekly/monthly/yearly frequency, minimum quantity, availability and lifecycle policies.
10 Quotation templates	Admin /admin/quotation-templates	Reusable products, plans, quantities, terms and validity days.
11 Discounts	Admin /admin/discounts	Fixed/percentage, min amount/quantity, dates, usage limit and product/subscription scope.
12 Taxes	Admin /admin/taxes	Tax name, rate, type, applicability and active status; preserve applied tax on invoices.
13 Subscriptions (MVP)	Admin /admin/subscriptions	Number, customer, plan, dates, next bill, amount, status; details, line items and valid lifecycle actions.
14 Billing (MVP)	Admin /admin/billing	Due, processed, failed and pending jobs; period, amount, status, invoice link and safe retry.
15 Invoices (MVP)	Admin /admin/invoices	Invoice number, customer, dates, subtotal, tax, discount, total, paid, balance and status; PDF/send/cancel as allowed.
16 Payments (MVP)	Admin /admin/payments	Record manual payment, method, amount, date, reference and allocate to invoice; partial payments.
17 Reports	Admin /admin/reports	Revenue, subscriptions, renewals, outstanding, overdue, payments and discount usage with date filters.
18 Staff management	Admin /admin/staff	Create/deactivate staff, role and permitted access; admin-only.
19 Settings	Admin /admin/settings	Company, currency, invoice numbering, payment terms, billing defaults, email and policy settings.
20 Staff dashboard (MVP)	Staff /staff/dashboard	Accessible customers, active subscriptions, invoices needing action, overdue, renewals and recent payments.
21 Staff customers (MVP)	Staff /staff/customers	Permitted customer search, profile, history and create quotation action.

ID / Page	Role / route	Required content and behavior
22 Staff quotations (MVP)	Staff /staff/quotations	Create, edit, send and convert quotations; customer, plan, products, quantity, taxes and validity.
23 Staff subscriptions (MVP)	Staff /staff/subscriptions	Create/view permitted subscriptions, valid lifecycle actions and renewal visibility.
24 Staff invoices/payments (MVP)	Staff /staff/invoices & payments	View permitted invoices, send/download, record payments and see balances.
25 Customer dashboard (MVP)	Customer /my/dashboard	Own active subscriptions, next renewal, outstanding balance, total paid and recent invoices.
26 My subscriptions (MVP)	Customer /my/subscriptions	Own plan, products, dates, frequency, amount, status and allowed requests.
27 Subscription detail	Customer /my/subscriptions/:id	Plan, price breakdown, next billing date, status and invoice history.
28 My invoices (MVP)	Customer /my/invoices	Own invoices with dates, amount, paid/balance, status and PDF download.
29 Invoice detail	Customer /my/invoices/:id	Business/customer details, line items, tax, discount, totals and download.
30 My payments	Customer /my/payments	Own payment references, invoice, date, amount, method and status.
31 Profile	Customer /my/profile	Edit permitted contact/billing details and change password.

7. Key Page Layout Details

7.1 Admin dashboard

- Top row: 4–6 KPI cards with label, value, small icon and link to details.
- Middle row: monthly revenue chart and subscription-status chart.
- Lower row: upcoming renewals table and recent activity feed.
- Default date range should be clear (e.g. current month); allow date filtering.
- Every metric must be calculated from database records, not hardcoded demo values.

7.2 Staff dashboard

- Show only records and metrics the logged-in Staff member is permitted to access.
- Prioritize operational tasks: quotations to send, renewals due, invoices overdue and payments to record.
- Avoid exposing global profit, cost price or company-wide sensitive data by default.

7.3 Customer dashboard

- Show the customer's own active subscriptions and next renewal information.
- Show outstanding balance and latest invoices with clear download links.
- Use plain-language status explanations and avoid internal accounting terminology where possible.

7.4 Tables and forms

- All list pages need search, pagination and relevant filters.
- Use empty states with a helpful next action; do not show fake records as real data.
- Use confirmation dialogs for destructive or financial actions.
- Show loading, success and error states for asynchronous actions.
- Use consistent status labels: Draft, Quotation, Confirmed, Active, Partially Paid, Paid, Overdue, Closed.

7.5 Invoice PDF

Use a clean printable invoice with company details, invoice number, dates, customer billing details, itemized lines, quantity, unit price, tax, discount, total, paid amount and balance due. The PDF must match the confirmed invoice record.

8. Core Business Workflows

8.1 Subscription lifecycle

```
Draft → Quotation → Confirmed → Active → Closed
Optional policy states: Paused / Renewed.
```

The backend must validate state transitions. Pausing, resuming, closing and renewing must follow configured business policies. A customer-facing request may require staff approval.

8.2 Quotation to subscription

- Staff selects a customer and optional quotation template.
- Staff adds plans/products, quantities and permitted discounts.
- Backend calculates subtotal, discount, tax and total.
- Quotation validity and terms are saved.
- Quotation is sent or shared with customer.
- After confirmation, create or activate the subscription using a defined conversion rule.

8.3 Recurring billing

- A scheduled job identifies active subscriptions whose next billing date is due.
- For each subscription, determine the exact billing period and validate eligibility.
- Calculate line items, applicable discounts and taxes.
- Create a unique billing record and invoice in one database transaction.
- Advance next billing date only when invoice creation succeeds.
- Queue invoice notification after transaction commit.
- Record success or failure with an error message and retry metadata.

8.4 Duplicate prevention

Create a database uniqueness constraint for subscription_id + billing_period_start + billing_period_end (or an equivalent deterministic billing key). Use transaction handling and row-level locking where appropriate. A retry must detect an already-created billing period and must not create a second invoice.

8.5 Invoice and payment statuses

Primary invoice workflow: Draft → Confirmed → Paid. Implementation may additionally use Partially Paid, Overdue and Cancelled. Overdue is based on due date and positive outstanding balance. Do not mark an invoice Paid until allocated payments cover the total.

8.6 Payment allocation

Support partial payments by recording payment transactions and allocating amounts to invoices. Example: invoice 10,000; payment one 4,000; payment two 6,000; remaining balance becomes zero. Never store raw card data.

9. Database and Data Model

Use PostgreSQL and SQLAlchemy. Model financial entities explicitly and enforce relationships with foreign keys and unique constraints.

Entity	Key fields / relationships
users	id, name, email (unique), password_hash, role, is_active, created_at
customers	id, user_id, company/name, phone, billing_address
products	id, name, SKU, type, sales_price, cost_price, notes, active
product_variants	id, product_id, attribute, value, price_extra, active
plans	id, name, price, billing_frequency, minimum_quantity, availability and policies
quotation_templates	id, name, validity_days, terms
quotations	id, customer_id, number, dates, status, totals
quotation_items	id, quotation_id, product_id/plan_id, description, quantity, unit_price
subscriptions	id, number, customer_id, plan_id, start/end dates, next_billing_date, status
subscription_items	id, subscription_id, product_id, description snapshot, quantity, unit price
discount_rules	id, type, value, minimums, validity, usage_limit, scope
discount_usages	id, discount_rule_id, invoice_id, customer_id, used_at
invoices	id, number, customer_id, subscription_id, dates, totals, amount_paid, balance, status
invoice_items	id, invoice_id, description snapshot, quantity, unit price, tax and discount snapshots
billing_records	id, subscription_id, billing_period_start/end, invoice_id, status, attempt/error
payments	id, customer_id, method, amount, date, reference, status
payment_allocations	id, payment_id, invoice_id, allocated_amount
audit_logs	id, actor_user_id, action, entity_type/id, timestamp, metadata

9.1 Data integrity rules

- Unique email addresses and unique invoice/subscription numbers.
- Use NUMERIC/DECIMAL for money, never binary floating-point.
- Store currency explicitly where relevant.
- Snapshot descriptions, unit prices, discounts and taxes on invoice lines.
- Use foreign keys and indexes for customer_id, status, due_date and billing-period lookups.
- Do not hard-delete financial records; use cancellation, archival or reversal workflows.
- Keep billing records unique per subscription and period.

10. API Design

Expose versioned REST endpoints under /api/v1. FastAPI should generate OpenAPI documentation. Use consistent response formats, pagination, validation and authorization.

Area	Example endpoints
Authentication	POST /auth/register, /auth/login, /auth/refresh, /auth/forgot-password, /auth/reset-password; GET /auth/me
Staff	POST /staff (Admin only); GET /staff; PATCH /staff/{id}
Products	GET/POST /products; GET/PATCH /products/{id}; POST /products/{id}/variants
Plans	GET/POST /plans; GET/PATCH /plans/{id}
Quotations	GET/POST /quotations; GET /quotations/{id}; POST /quotations/{id}/confirm
Subscriptions	GET/POST /subscriptions; GET /subscriptions/{id}; POST /subscriptions/{id}/activate|pause|resume|close|renew
Billing	GET /billing/history; POST /billing/run (restricted/admin operation)
Invoices	GET /invoices; GET /invoices/{id}; POST /invoices/{id}/confirm; GET /invoices/{id}/pdf
Payments	GET/POST /payments; POST /payments/{id}/allocations
Analytics	GET /analytics/overview; /revenue; /overdue; /subscriptions
Customer portal	GET /my/subscriptions; GET /my/invoices; GET /my/payments; GET/PATCH /my/profile

API rules

- Use Pydantic request/response schemas.
- Return 401 for unauthenticated requests and 403 for forbidden actions.
- Use ownership checks for every customer-specific resource.
- Paginate list endpoints and validate sort/filter parameters.
- Use idempotency keys for externally retried payment or billing operations where appropriate.
- Never return password hashes, secrets or internal-only cost fields to customers.

11. Security, Reliability and Performance

11.1 Security

- Hash passwords with Argon2id.
- Use short-lived access tokens and secure refresh-token rotation/revocation.
- Apply RBAC and customer ownership checks server-side.
- Validate all input and use parameterized ORM queries.
- Use HTTPS in production and keep secrets out of source control.
- Use expiring, single-use password reset tokens; store token hashes.
- Rate-limit login and password-reset endpoints.
- Audit important financial and administrative actions.
- Back up PostgreSQL and test restoration.

11.2 Reliability

- Create billing record and invoice atomically.
- Use database uniqueness constraints to prevent duplicate billing.
- Use retry-safe background jobs and record failure reasons.
- Send notifications after the financial transaction commits.
- Do not advance the next billing date if invoice creation fails.
- Protect discount usage limits against concurrent requests.

11.3 Performance

- Use pagination and indexed filters; never fetch every subscription for a dashboard.
- Aggregate KPIs in database queries.
- Move recurring billing, PDF batches and bulk emails out of request/response handlers.
- Use batch processing and avoid N+1 queries.
- Target common dashboard/list interactions under two seconds, then verify with realistic load tests.
- Add Redis, connection pooling or reporting replicas only when measurements justify them.

12. Hackathon Execution Plan and Demo Story

12.1 Suggested team workstreams

Workstream	Responsibility
Frontend	React layouts, role-based navigation, forms, tables, dashboards and customer portal
Backend	FastAPI endpoints, auth, validation, business services and permissions
Database / billing	PostgreSQL schema, migrations, subscription lifecycle, billing idempotency and payments
Integration / QA	API integration, demo data, PDF, email, test scenarios and deployment

A small team can combine roles. Agree on API contracts and model names before parallel implementation.

12.2 Recommended build order

Priority	Build	Demo outcome
P0	Login + roles + database	Three users land on different workspaces
P0	Products + plans + customer	Admin configures catalogue and customer
P0	Subscription creation	Staff creates and activates a subscription
P0	Recurring billing + invoice	Due subscription generates one invoice
P0	Payment recording	Staff records partial/full payment and balance updates
P1	Customer portal	Customer sees own subscription and invoice PDF
P1	Dashboards + filters	Admin sees revenue, active subscriptions and overdue invoices
P2	Email, advanced discounts, reports	Polish and additional capability

12.3 Suggested live demo script

- Admin logs in and creates a product and monthly plan.
- Admin creates a customer or Staff registers one.
- Staff creates a quotation and converts it into a subscription.
- Trigger the billing job for a due subscription.
- Show the generated invoice and billing-period record.
- Run billing again and demonstrate that no duplicate invoice is created.
- Record a partial payment, then a second payment; show the balance reaching zero.
- Log in as the customer and show only that customer's subscription, invoice and payment history.
- Return to Admin dashboard and show updated metrics.

12.4 What judges should notice

- Clear business problem and measurable workflow improvement.
- End-to-end working flow, not just static dashboard screens.
- Duplicate-safe recurring billing and correct financial calculations.
- Different interfaces and permissions for three user types.
- Scalable, maintainable architecture without unnecessary microservices.
- A credible roadmap for payment gateway and enterprise features.

13. Testing and Acceptance Criteria

13.1 Critical tests

- Authentication: valid/invalid login, duplicate email, password policy and reset token expiry.
- Authorization: Staff cannot create Staff; Customer cannot access another customer's records.
- Subscription: valid/invalid transitions, pause/resume/close and renewal dates.
- Billing: daily/weekly/monthly/yearly periods, concurrent due subscriptions, retry and duplicate prevention.
- Finance: tax, fixed/percentage discount, discount expiry/limit, partial and full payments.
- Invoices: immutable confirmed totals, cancellation rules and PDF consistency.
- Performance: realistic dataset with thousands of subscriptions and paginated invoice lists.

13.2 Acceptance criteria

- Admin can create Staff users and configure products/plans.
- Customer registration creates Customer-only accounts.
- Staff can create quotations and subscriptions within permissions.
- Due active subscriptions generate invoices automatically.
- The same subscription period cannot generate duplicate invoices.
- Tax and discount totals are correct and traceable.
- Partial payments update invoice balances correctly.
- Customers can access only their own data.
- Dashboards show database-backed values.
- The app can be started from documented setup instructions.

13.3 Final deliverables

- Frontend source code and backend source code.
- PostgreSQL schema and migrations.
- Docker Compose and environment example.
- API documentation.
- Admin, Staff and Customer experiences.
- Billing scheduler and invoice PDF generation.
- Automated tests and demo seed data.
- README with installation, configuration, run and deployment steps.
- Short demo video or presentation, if required by the hackathon.

14. Master Prompt for an AI Coding Agent

Copy the prompt below into your coding agent together with this specification. Ask it to implement one phase at a time and inspect the repository before modifying files.

```
You are a senior full-stack engineer. Build SubNexa, a hackathon-ready subscription management and recurring billing SaaS application, using the attached project specification as the source of truth. TECH STACK: - Frontend: React 18 + Vite + Tailwind CSS + shadcn/ui + Lucide React + Recharts. - Backend: Python + FastAPI + Pydantic + SQLAlchemy + Alembic. - Database: PostgreSQL. - Authentication: JWT access/refresh tokens and Argon2id password hashing. - Background billing: APScheduler for a simple MVP; use Celery + Redis only if the environment and schedule justify it. - PDFs: ReportLab. - Deployment: Docker Compose. - Tests: Pytest, HTTPX and Playwright. RULES: 1. Do not use Firebase, Supabase or another backend-as-a-service. 2. Do not introduce Odoo as a second backend. This specification chooses a custom stack for hackathon speed. Odoo may be considered only as a separate alternative if explicitly requested. 3. Use a modular monolith, not microservices. 4. PostgreSQL is the source of truth. 5. Enforce Admin, Staff and Customer permissions on the backend. 6. Customers must only access their own records. 7. Use decimal-safe financial calculations and invoice snapshots. 8. Prevent duplicate billing for the same subscription and billing period with database constraints and transaction-safe logic. 9. Support partial payments and accurate outstanding balances. 10. Use the supplied UI colors, typography and role-specific page requirements. 11. Do not hardcode dashboard metrics or pretend that unfinished buttons work. 12. Include migrations, validation, tests, error handling and setup documentation. IMPLEMENTATION PROCESS: - First inspect the repository and report the existing structure. - Confirm Python, Node and PostgreSQL versions available. - Propose a phase-by-phase plan and database relationships. - Implement Phase 1 only: project setup, database connection, migrations, user model, authentication and RBAC. - Provide complete files with paths and exact run commands. - Run tests and fix errors before moving to the next phase. - Wait for approval before proceeding to the next phase. - Continue through catalogue, quotations, subscriptions, billing, invoices, payments, customer portal, analytics and deployment. - If any requirement is ambiguous or conflicts with the source specification, explain the ambiguity and ask before making a business-critical assumption. The goal is a working, demonstrable hackathon MVP with reliable subscription billing, not merely a collection of attractive static screens.
```

End of specification

Project: SubNexa – Intelligent Subscription Management and Recurring Billing System